Trabalho Prático: Divisão e Conquista

Dupla: **Matheus Cruz** e **Enzo Fernandes** · Linguagem: Java · Outubro de 2026

1. Introdução e ambiente

Implementamos dois problemas resolvidos por divisão e conquista: o **subarranjo de soma máxima** (Etapa 1) e o **par de pontos mais próximos** (Etapa 2). Cada um tem uma versão de força bruta $O(n^2)$ e uma de divisão e conquista O(n log n); na Etapa 1 implementamos também o algoritmo de Kadane, O(n), como questão desafio. Todas as versões foram validadas por uma bateria de testes e medidas experimentalmente (Etapa 3). O código, os casos de teste, os CSVs com as medições brutas e as instruções de execução estão no repositório (ver `README.md`).

Máquina usada nas medições: notebook com Intel Core i7-12700H (14 núcleos, 20 threads, até 4,7 GHz), 16 GB de RAM, Windows 11 Pro 64 bits, OpenJDK 25.0.4 (HotSpot 64-Bit Server VM), executado com `-Xmx6g`. Os programas são de uma única thread.

2. Etapa 1 — Subarranjo de soma máxima

Entrada: n e uma sequência de n inteiros (variações diárias). **Saída:** a soma máxima e os índices de início e fim (base 0, intervalo fechado). Convenções adotadas por todas as implementações: o subarranjo nunca é vazio (se todos os valores forem negativos, a resposta é o maior elemento) e, em caso de **empate**, vence o intervalo de menor início e, persistindo o empate, o de menor fim. Os acumuladores são de 64 bits (`long`), pois a soma de valores `int` pode passar de $2^{31}-1$.

2.1 Força bruta

Para cada início i, percorre os fins j ≥ i acumulando a soma, de forma que cada intervalo custa O(1). Atualiza a melhor resposta só quando a soma é *estritamente* maior, o que mantém o primeiro intervalo na ordem (i, j) e implementa a regra de desempate. O número de intervalos testados é

$$T(n) = \Sigma_{i=0}^{n-1} (n - i) = n(n+1)/2 = \Theta(n^2).$$

2.2 Divisão e conquista

Divide o vetor `a[lo..hi]` no meio `mid`, resolve recursivamente as duas metades e calcula, em O(n), o melhor subarranjo que cruza o meio: o melhor sufixo de `a[lo..mid]` (varrendo de `mid` para a esquerda) somado ao melhor prefixo de `a[mid+1..hi]`. A resposta é a melhor das três candidatas. Como a escolha do sufixo e a do prefixo são independentes, basta, em empates, ficar com o sufixo que começa mais à esquerda (comparação `>=`) e com o prefixo que termina mais à esquerda (`>`); a comparação final entre as três candidatas usa a mesma regra (maior soma, menor início, menor fim). A recorrência é

$$T(1) = \Theta(1), T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n).$$

Pelo Teorema Mestre, com a = 2, b = 2 e f(n) = Θ(n) = $\Theta(n^{\log_2 2})$, estamos no caso 2, logo **T(n) = Θ(n log n)**. Pela árvore de recursão: há $\log_2 n$ níveis e em cada nível os subproblemas somam n elementos, com custo cn por nível, totalizando $cn \cdot \log_2 n$.

2.3 Questão desafio — Kadane, O(n)

Mantém `cur`, a melhor soma de um subarranjo que termina na posição j. Se `cur ≥ 0`, vale a pena estender o subarranjo anterior; senão, começa-se um novo em j. A melhor resposta é atualizada quando `cur` fica estritamente maior que ela. Estender também quando `cur = 0` mantém o início mais à esquerda e, com isso, a mesma regra de desempate das outras versões (conferido nos testes aleatórios). Recorrência: T(n) = T(n−1) + Θ(1) = **Θ(n)**. É uma programação dinâmica: a resposta "terminando em j" é obtida a partir da resposta "terminando em j−1".

3. Etapa 2 — Par de pontos mais próximos

Entrada: n e as coordenadas reais (x, y) de n estações. **Saída:** a menor distância euclidiana e os índices dos dois pontos. Para evitar raízes quadradas, as comparações usam a distância ao quadrado.

3.1 Força bruta

Compara todos os pares: n(n−1)/2 comparações, isto é, **$\Theta(n^2)$**.

3.2 Divisão e conquista

1. **Pré-processamento (uma vez):** ordenar os índices dos pontos por x (`Px`) e por y (`Py`) com o `Arrays.sort` da biblioteca padrão — Θ(n log n). Cada ponto guarda sua posição (*rank*) em `Px`.
2. **Dividir:** a linha vertical passa pela mediana das coordenadas x, x_m = `Px[n/2].x`. A metade esquerda são os pontos com rank < n/2. Separar por rank (e não por x) garante metades corretas mesmo quando muitos pontos têm x igual ao da mediana. `Py` é particionado em Py_E e Py_D em O(n), preservando a ordem por y, sem nova ordenação.
3. **Conquistar:** resolver cada lado recursivamente, obtendo $\delta = \min(\delta_E, \delta_D)$.
4. **Combinar:** montar a faixa com os pontos de `Py` com $|x - x_m| < \delta$ (já ordenados por y) e comparar cada ponto apenas com os **até 7 seguintes**, parando antes se a diferença em y chegar a δ.
5. **Limiar:** subproblemas com até **32** pontos são resolvidos por força bruta (seção 6.3).

Cada chamada faz trabalho linear (particionar `Py`, montar a faixa e no máximo 7m comparações para uma faixa de m ≤ n pontos), então

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n), \text{ e o total é } \Theta(n \log n)\ [\text{ordenação}] + \Theta(n \log n)\ [\text{recursão}] = \Theta(n \log n).$$

Com limiar t, a recursão para em n/t folhas de custo $\Theta(t^2)$ cada: o custo vira $\Theta(n \cdot t) + \Theta(n \cdot \log(n/t))$, que continua Θ(n log n) para t constante.

3.3 Por que bastam 7 comparações por ponto na faixa

> Seja p um ponto da faixa e considere os pontos q que vêm depois dele na ordem por y e que ainda podem estar a distância < δ: eles têm $y_p \le y_q < y_p + \delta$ e $|x_q - x_m| < \delta$. Todos estão, portanto, no retângulo R de largura 2δ e altura δ com base em y_p e centro na linha divisória. Divida R em 8 quadrados de lado δ/2 (4 de cada lado da linha). Cada quadrado contém **no máximo um ponto**: dois pontos no mesmo quadrado estariam do mesmo lado da linha e a uma distância de no máximo a diagonal $\delta/\sqrt{2} < \delta$, contradizendo o fato de que δ é menor ou igual à menor distância entre pontos de um mesmo lado. (Pontos exatamente sobre a linha pertencem a um único lado, pelo rank, e cada lado usa só os seus 4 quadrados.) Logo R contém no máximo 8 pontos, sendo um deles o próprio p; restam **no máximo 7** candidatos depois de p na ordem por y. Qualquer ponto mais adiante na lista tem $y \ge y_p + \delta$ e não pode melhorar a resposta. Pares em que q vem antes de p já foram examinados quando q foi o ponto de referência.

4. Testes

O programa `TestRunner` executa duas baterias e termina com código 0 apenas se tudo passar:

1. **Casos fixos** (`tests/casos_etapa1.txt` e `tests/casos_etapa2.txt`), cada um com entrada, saída esperada calculada à mão e justificativa. Todas as implementações rodam em todos os casos. Na Etapa 2, a divisão e conquista roda com limiares 3, 4, 8 e 32, para que até os casos pequenos passem pela recursão e pela faixa, em vez de caírem direto na força bruta.
2. **Testes aleatórios contra a força bruta:** 3.000 vetores (n de 1 a 300, com valores em faixas estreitas para provocar muitos empates e muitos zeros, e às vezes quase só negativos), exigindo soma *e índices* idênticos;

e 2.000 conjuntos de pontos (n de 2 a 600, metade com coordenadas inteiras pequenas, o que gera x repetidos, pontos duplicados e empates de distância), com limiares de 3 a 32.

Resultado: **8.120 verificações, 0 falhas**. Casos de borda e justificativas:

Caso (Etapa 1)	Por que é importante
tamanho 1 (positivo, negativo, zero)	Caso base da recursão; com valor negativo a resposta não pode ser o vetor vazio de soma 0.
todos negativos (com repetição), todos iguais	A resposta é o maior elemento; o máximo aparece duas vezes e deve vencer o primeiro.
Integer.MIN_VALUE; soma > $2^{31}-1$	Detecta estouro: o acumulador precisa ser de 64 bits.
todos positivos; todos zeros; dois elementos	Vetor inteiro como resposta; empate total; menor caso que divide.
exemplos clássicos; cruza o meio; resposta no fim	Garantem que a combinação (subarranjo que cruza o meio) e as bordas estão corretas.
empates: disjuntos, três regiões, mesmo início, mesmo fim, aninhados, cruzando × lado	Exercitam a regra de desempate em cada ponto onde ela pode falhar: dentro de uma metade, no sufixo, no prefixo e na comparação final das três candidatas.

Caso (Etapa 2)	Por que é importante
dois pontos; três colineares	Menor entrada válida; empate entre pares.
duplicados; todos iguais	Distância zero; muitos pontos na mesma posição.
mesma vertical; mesma horizontal	Todos com o x da mediana (divisão precisa usar o rank); faixa toda empatada em y.
par cruza a linha; duas colunas intercaladas	A resposta só é encontrada na faixa — testam a etapa de combinação.
grade 4×4	Muitos empates de distância e muitos pontos simultâneos na faixa (exercita o limite de 7).
negativos/decimais; coordenadas ~10^9; par isolado no canto	Precisão numérica e par mais próximo longe da mediana.

5. Metodologia experimental

O programa `Benchmark` mede cada algoritmo em tamanhos que dobram a cada passo (100, 200, …, 6.400, como no enunciado, e depois tamanhos maiores, até 1.638.400 para os algoritmos rápidos). Para cada tamanho são feitas **10 execuções**, cada uma com uma entrada aleatória nova (semente fixa): inteiros uniformes em [−1000, 1000] na Etapa 1 e pontos uniformes no quadrado $[0, 10^6)^2$ na Etapa 2. Antes das medições o JIT é aquecido. Quando uma chamada leva menos de 2 ms, ela é repetida até somar 2 ms e registra-se o tempo médio por chamada. Os tempos brutos estão em `resultados/tempos.csv`, `limiar.csv` e `cruzamento.csv`. As tabelas mostram média ± desvio padrão das 10 execuções e a razão T(n)/T(n/2): pela análise, ela deve tender a **4** para $\Theta(n^2)$, a **pouco mais de 2** para $\Theta(n \log n)$ ($2\cdot\log(n)/\log(n/2)$) e a **2** para $\Theta(n)$.

6. Resultados

6.1 Etapa 1 — subarranjo de soma máxima

Tabela 1. Tempo médio por execução (10 execuções por tamanho) e razão entre tamanhos consecutivos.

n	Força bruta média ± desvio	T(n)/T(n/2)	Divisão e conquista média ± desvio	T(n)/T(n/2)	Kadane (O(n)) média ± desvio	T(n)/T(n/2)
100	1,15 µs ± 160 ns	—	1,58 µs ± 507 ns	—	102 ns ± 39 ns	—
200	4,74 µs ± 129 ns	4,12	2,94 µs ± 762 ns	1,86	124 ns ± 19 ns	1,22
400	19,7 µs ± 350 ns	4,16	6,51 µs ± 1,88 µs	2,21	189 ns ± 13 ns	1,52
800	77,1 µs ± 2,98 µs	3,92	12,6 µs ± 2,25 µs	1,93	338 ns ± 16 ns	1,79
1.600	303 µs ± 11,7 µs	3,94	24,7 µs ± 3,26 µs	1,96	642 ns ± 29 ns	1,90
3.200	1,31 ms ± 118 µs	4,32	52,5 µs ± 5,24 µs	2,13	1,33 µs ± 87 ns	2,07
6.400	4,78 ms ± 130 µs	3,65	108 µs ± 12,6 µs	2,06	2,43 µs ± 187 ns	1,83

Tabela 2. Tamanhos maiores (a força bruta foi medida até 51.200).

n	Força bruta média ± desvio	T(n)/T(n/2)	Divisão e conquista média ± desvio	T(n)/T(n/2)	Kadane (O(n)) média ± desvio	T(n)/T(n/2)
12.800	19,2 ms ± 464 µs	4,02	236 µs ± 28,8 µs	2,18	4,49 µs ± 257 ns	1,84
25.600	77,5 ms ± 1,17 ms	4,04	461 µs ± 35,8 µs	1,96	8,45 µs ± 354 ns	1,88
51.200	312 ms ± 3,7 ms	4,02	1,04 ms ± 136 µs	2,25	17,1 µs ± 954 ns	2,02
102.400	—	—	2,12 ms ± 206 µs	2,04	46,9 µs ± 2,81 µs	2,75
204.800	—	—	4,52 ms ± 451 µs	2,13	88,5 µs ± 2,14 µs	1,89
409.600	—	—	9,81 ms ± 570 µs	2,17	207 µs ± 16,4 µs	2,34
819.200	—	—	20,6 ms ± 1,08 ms	2,10	389 µs ± 39,7 µs	1,88
1.638.400	—	—	41,7 ms ± 778 µs	2,03	802 µs ± 123 µs	2,06

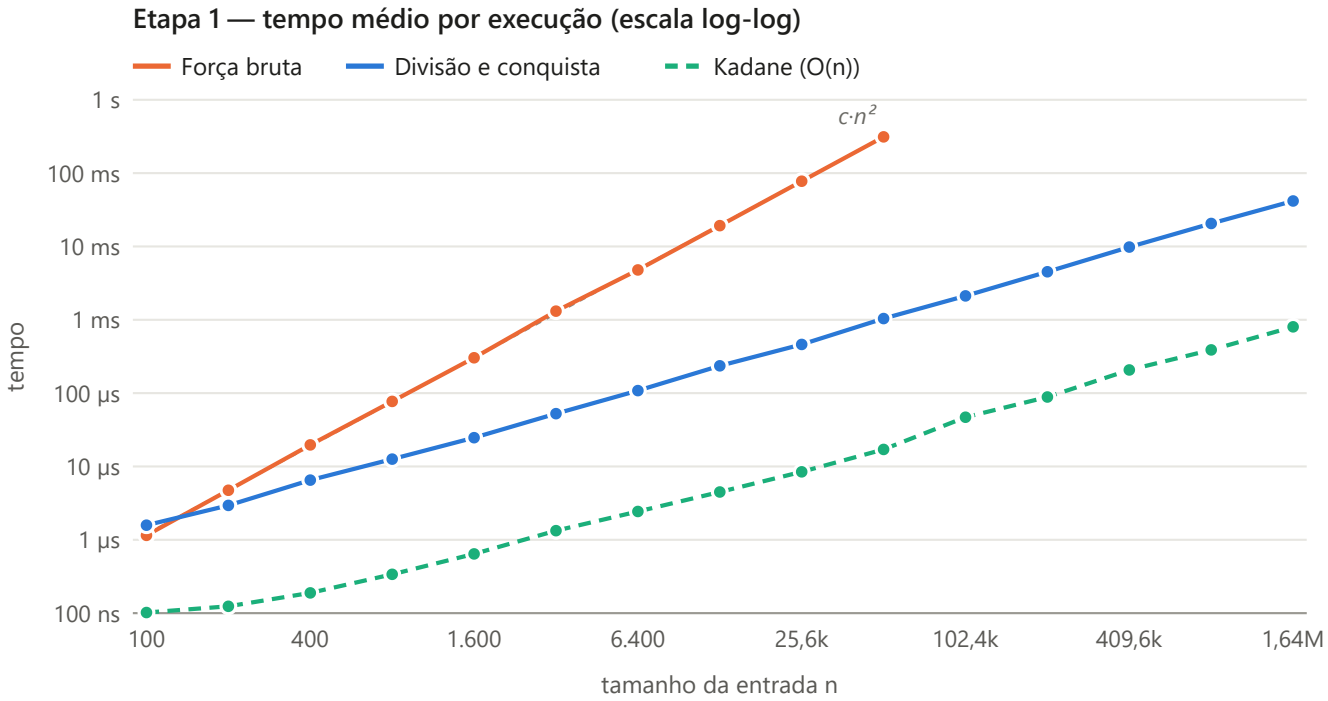


Figura 1. Etapa 1 em escala log-log: uma reta de inclinação k indica tempo ∝ n^k. A linha tracejada cinza é $c \cdot n^2$ ajustada ao último ponto da força bruta.

6.2 Etapa 2 — par de pontos mais próximos

Tabela 3. Tempo médio por execução; a divisão e conquista inclui a ordenação inicial e usa limiar 32.

n	Força bruta média ± desvio	T(n)/T(n/2)	Divisão e conquista média ± desvio	T(n)/T(n/2)
100	2,72 µs ± 179 ns	—	6,24 µs ± 304 ns	—
200	10,8 µs ± 588 ns	3,95	17,2 µs ± 6,54 µs	2,75
400	43,5 µs ± 2,82 µs	4,04	68,8 µs ± 5,6 µs	4,00
800	162 µs ± 4,39 µs	3,73	200 µs ± 9,43 µs	2,91
1.600	661 µs ± 47,6 µs	4,08	452 µs ± 13,5 µs	2,26
3.200	2,56 ms ± 128 µs	3,87	1,01 ms ± 45,3 µs	2,22
6.400	10,4 ms ± 227 µs	4,07	2,34 ms ± 203 µs	2,32

Tabela 4. Tamanhos maiores (a força bruta foi medida até 25.600).

n	Força bruta média ± desvio	T(n)/T(n/2)	Divisão e conquista média ± desvio	T(n)/T(n/2)
12.800	41,6 ms ± 1,16 ms	3,99	5,01 ms ± 182 µs	2,15
25.600	167 ms ± 3,4 ms	4,02	10,7 ms ± 278 µs	2,14
51.200	—	—	23,2 ms ± 651 µs	2,17
102.400	—	—	50,3 ms ± 1,33 ms	2,17
204.800	—	—	114 ms ± 2,23 ms	2,27
409.600	—	—	263 ms ± 10,6 ms	2,30
819.200	—	—	594 ms ± 7,54 ms	2,26
1.638.400	—	—	1,43 s ± 21,1 ms	2,40

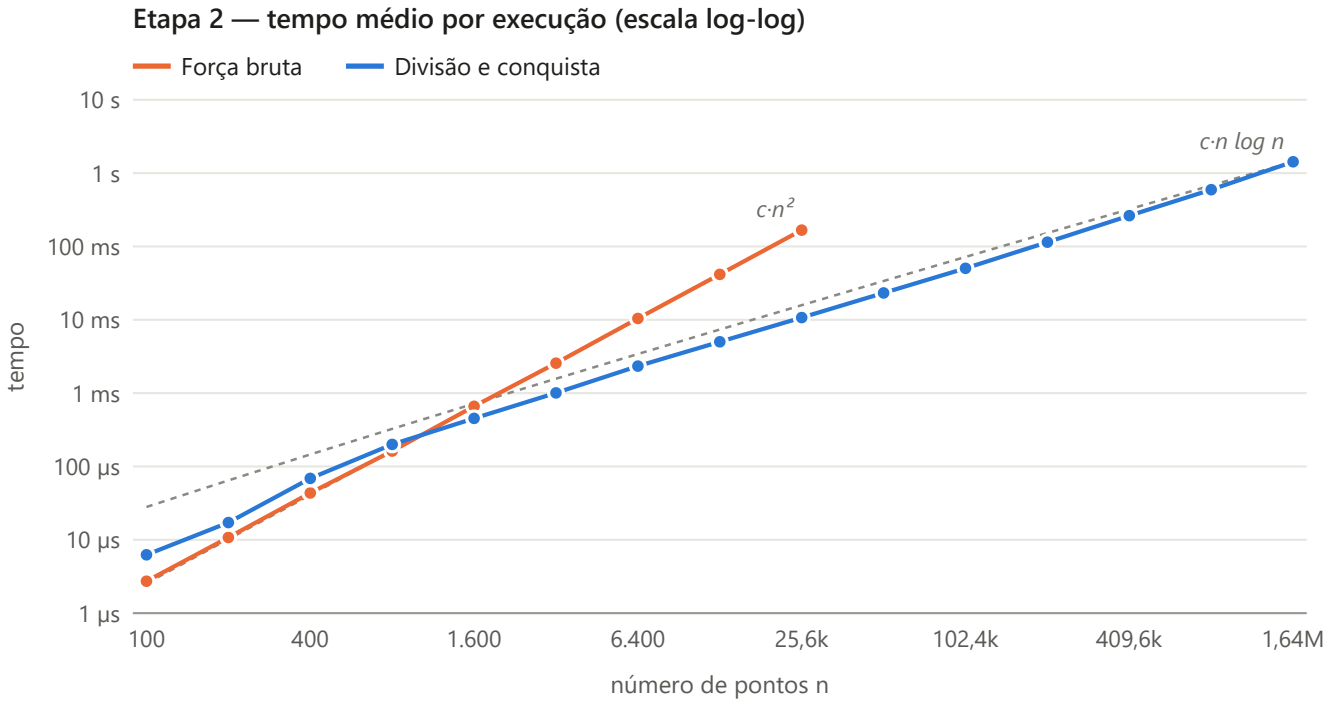


Figura 2. Etapa 2 em escala log-log, com as curvas c·n² e c·n log n ajustadas ao último ponto de cada algoritmo.

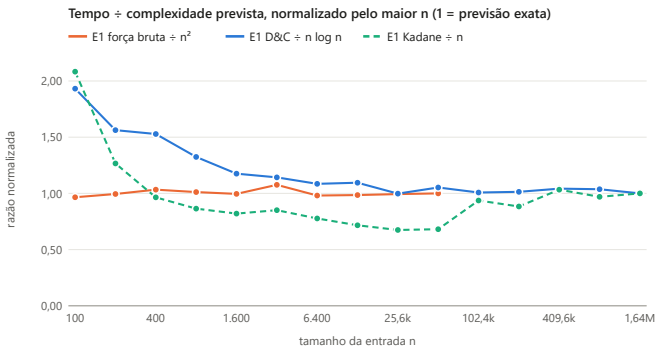


Figura 3. Etapa 1: tempo dividido pela complexidade prevista. Linha horizontal = previsão confirmada.

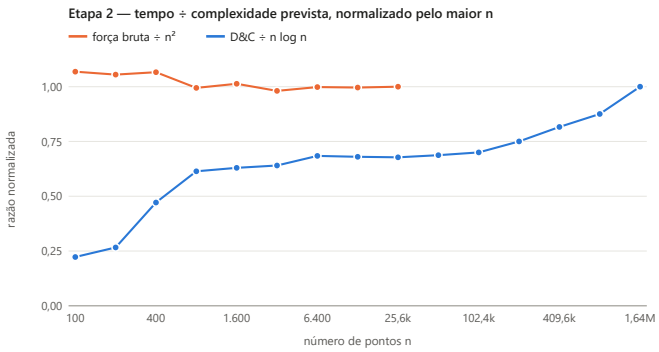


Figura 4. O mesmo para a Etapa 2.

Tabela 5. Divisão do tempo da divisão e conquista da Etapa 2 entre a ordenação inicial e a fase recursiva (média de 20 execuções, `resultados/limiar.csv`).

n	ordenação inicial	recursão (limiar 32)	fração da ordenação
1.000	231 µs	23,1 µs	91%
10.000	3,26 ms	597 µs	85%
100.000	39,8 ms	8,68 ms	82%
1.000.000	562 ms	175 ms	76%

6.3 Escolha do limiar da Etapa 2

Medimos só a fase recursiva (a ordenação inicial não depende do limiar) para 13 limiares entre 3 (recursão até o fim, sem atalho) e 256, com n = 1.000, 10.000, 100.000 e 1.000.000, 20 execuções cada. A Figura 5 mostra o tempo de cada limiar dividido pelo melhor tempo obtido para aquele n.

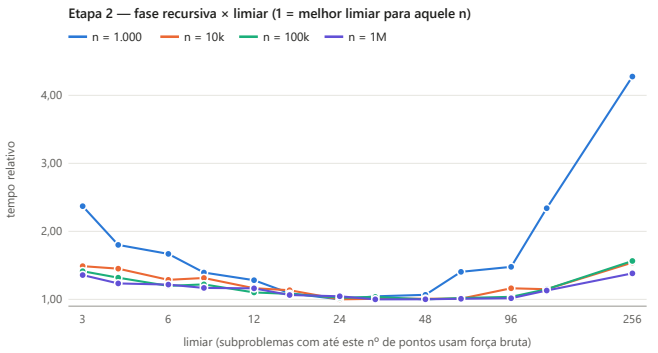


Figura 5. Fase recursiva da Etapa 2 em função do limiar (1 = melhor limiar para o n).

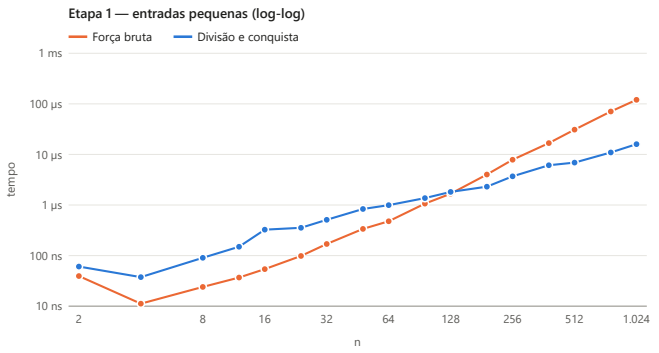


Figura 6. Etapa 1 em entradas pequenas: as curvas se cruzam entre n = 128 e n = 192.

7. Discussão

7.1 Comportamento observado × previsto

- **Forças brutas (Θ(n²)).** Nas duas etapas a razão T(n)/T(n/2) fica entre 3,7 e 4,3, em torno de 4, e a inclinação da reta log-log ajustada (n ≥ 1.600) é **1,99** na Etapa 1 e **2,00** na Etapa 2. Nas Figuras 3 e 4 a curva tempo/n^2 é praticamente horizontal. A previsão quadrática se confirma com folga, o que é esperado: são dois laços aninhados sobre vetores contíguos, sem alocação, e o custo por iteração é constante.
- **Divisão e conquista da Etapa 1 (Θ(n log n)).** As razões ficam entre 1,86 e 2,25 (média ≈ 2,07), compatíveis com o valor teórico 2·log(n)/log(n/2), que vai de ≈ 2,15 (n = 200) a ≈ 2,05 (n = 1,6 milhão). A inclinação ajustada é 1,08 — maior que 1 por causa do fator log n, mas longe de 2. Na Figura 3, a curva tempo/(n log n) começa alta (custos fixos das chamadas recursivas pesam mais em n pequeno) e fica aproximadamente horizontal a partir de n ≈ 1.600.

- **Kadane (Θ(n)).** Razões próximas de 2 e inclinação 1,04. Em n pequeno o tempo (≈ 100 ns) é dominado por custos fixos da chamada, por isso as primeiras razões ficam abaixo de 2 e a curva normalizada da Figura 3 começa acima de 1. Em n = 102.400 há um degrau (razão 2,75): o custo por elemento sobe cerca de 40% e se mantém nos tamanhos seguintes, o que indica um efeito de memória (o vetor sai dos níveis mais rápidos de cache), e não uma mudança de complexidade — depois do degrau as razões voltam para ≈ 2.
- **Divisão e conquista da Etapa 2 (Θ(n log n)).** A partir de n = 1.600 as razões ficam entre 2,14 e 2,40, inclinação 1,15. Na Figura 4, a curva tempo/(n log n) fica quase horizontal entre n = 1.600 e n = 102.400 (de 0,65 a 0,75) e sobe para 1 nos maiores tamanhos. A causa é a hierarquia de memória, que a análise assintótica não modela: com mais de 400 mil pontos os vetores auxiliares (coordenadas, índices, ranks e chaves de ordenação) passam de dezenas de MB e excedem o cache L3 de 24 MB, e os acessos indiretos (`xs[Py[k]]`, buscas binárias da ordenação) passam a ir à memória principal. Em n ≤ 800 a curva também sobe e as razões oscilam (2,75; 4,00; 2,91): são tempos de poucas dezenas de microssegundos, em que a mistura de custos muda com n (folhas de força bruta, e o `Arrays.sort` usa ordenação por inserção para vetores pequenos e outro algoritmo para os maiores).

A Tabela 5 mostra outro ponto: a **ordenação inicial** responde por 76% a 91% do tempo da divisão e conquista da Etapa 2. A fase recursiva (a parte que o enunciado pede para focar) é rápida: para 1 milhão de pontos leva 0,18 s, contra 0,56 s da ordenação. As duas fases são Θ(n log n), mas a ordenação tem constante maior, porque ordena índices duas vezes (por x e por y) e precisa de buscas binárias para montar as chaves.

7.2 Por que a força bruta vence em entradas pequenas e a técnica vence nas grandes

Em notação assintótica, $c_1 n^2 > c_2 n \log n$ para n grande quaisquer que sejam as constantes, mas para n pequeno são as constantes que decidem. A força bruta tem a menor constante possível: laços simples, acesso sequencial à memória, nenhuma chamada de método e nenhuma alocação. A divisão e conquista paga, a cada nível, chamadas recursivas, cálculo do meio e comparação de três candidatas (Etapa 1) ou alocação dos vetores Py_E/ Py_D, montagem da faixa e raízes quadradas (Etapa 2), além da ordenação inicial na Etapa 2.

- **Etapa 1:** a força bruta é mais rápida até n ≈ 128 (1,7 µs contra 1,8 µs) e perde a partir de n ≈ 192 (4,0 µs contra 2,3 µs) — Figura 6. Em n = 6.400 a divisão e conquista já é 44 vezes mais rápida (108 µs contra 4,8 ms) e em n = 51.200, 300 vezes. O Kadane é ainda mais rápido em todos os tamanhos medidos, pois tem a constante pequena da força bruta *e* complexidade linear.
- **Etapa 2:** o cruzamento acontece entre n = 800 (162 µs contra 200 µs, força bruta à frente) e n = 1.600 (661 µs contra 452 µs). Ele acontece bem mais tarde que na Etapa 1 porque a constante da divisão e conquista inclui a ordenação inicial, que sozinha custa cerca de 0,23 ms para 1.000 pontos. Daí em diante a vantagem cresce rapidamente: 4,5 vezes em n = 6.400 e 15,6 vezes em n = 25.600 (167 ms contra 10,7 ms). Extrapolando a curva quadrática para 1,6 milhão de pontos, a força bruta levaria cerca de 11 minutos; a divisão e conquista levou 1,4 s.

7.3 Justificativa do limiar escolhido (Etapa 2)

O limiar t equilibra dois custos. Resolver as folhas por força bruta custa cerca de $(n/t)\cdot t^2/2 = n\cdot t/2$ comparações, que crescem linearmente com t. Cada nível de recursão eliminado economiza um passo de combinação de custo linear, que é caro na prática (chamada recursiva, alocação e preenchimento de Py_E e Py_D, varredura para montar a faixa, raízes quadradas). Um limiar maior elimina cerca de $\log_2 t$ níveis. Para t pequeno, cada comparação extra na folha é muito mais barata que o nível de recursão que ela evita; para t grande, o custo quadrático das folhas passa a dominar.

Os dados confirmam esse comportamento em formato de "U" (Figura 5). Sem limiar (t = 3), a fase recursiva é de 36% a 137% mais lenta que no melhor caso. Com t = 128 ou 256 o custo quadrático das folhas aparece (até 4,3 vezes mais lento para n = 1.000). Entre 24 e 48 há um platô: a média geométrica do tempo relativo nos quatro tamanhos é 1,017 (t = 24), 1,022 (t = 32) e 1,019 (t = 48), diferenças menores que o ruído das medições. Escolhemos **t = 32**, no centro do platô: em todos os quatro tamanhos ele fica a no máximo 4% do melhor limiar medido, e fica longe das duas bordas do "U", onde o desempenho piora rápido. Note que o limiar não

muda a complexidade (é uma constante); ele só melhora a constante, reduzindo o tempo da fase recursiva em 26% a 56% em relação à recursão completa.

8. Conclusão

As cinco implementações passaram em todos os testes, incluindo os casos de borda pedidos (somente negativos, tamanho 1 e empates) e os testes aleatórios contra a força bruta. Os experimentos confirmaram as complexidades obtidas pelas recorrências: $\Theta(n^2)$ para as forças brutas (razão ≈ 4 ao dobrar n, inclinação 2,0), $\Theta(n \log n)$ para as duas divisões e conquistas (razão ≈ 2,1–2,3) e $\Theta(n)$ para o Kadane (razão ≈ 2). Os desvios observados vêm de fatores que a análise assintótica ignora — custos fixos em entradas pequenas e cache em entradas grandes —, e não mudam a ordem de crescimento. A força bruta só vale a pena em entradas muito pequenas (até ~130 elementos na Etapa 1 e ~1.000 pontos na Etapa 2); essa mesma observação justifica usar força bruta dentro da recursão da Etapa 2 para subproblemas com até 32 pontos.